

# Practical: Matrix Multiplication using CUDA C

## Laboratory Practice V – Computer Engineering

### 1 Aim

Write a Program for Matrix Multiplication using CUDA C.

#### 1.1 Objective

Students should be able to perform Matrix Multiplication using CUDA C on GPU.

### 2 Theory

#### 2.1 What is CUDA?

CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by **NVIDIA**. It allows developers to use the power of NVIDIA GPUs to accelerate computation.

| Feature  | CPU              | GPU (CUDA)                   |
|----------|------------------|------------------------------|
| Cores    | 4–16             | 1000+                        |
| Best for | Sequential tasks | Parallel tasks               |
| Threads  | Few              | Thousands                    |
| Example  | File I/O, logic  | Matrix ops, image processing |

#### 2.2 CUDA Terminology

| Term                         | Meaning                                               |
|------------------------------|-------------------------------------------------------|
| <b>Host</b>                  | CPU and its memory (RAM)                              |
| <b>Device</b>                | GPU and its memory (VRAM)                             |
| <b>Kernel</b>                | Function that runs on GPU ( <code>__global__</code> ) |
| <b>Thread</b>                | Smallest unit of execution on GPU                     |
| <b>Block</b>                 | Group of threads                                      |
| <b>Grid</b>                  | Group of blocks                                       |
| <code>threadIdx.x, .y</code> | Thread's position within its block                    |
| <code>cudaMalloc</code>      | Allocate memory on GPU                                |
| <code>cudaMemcpy</code>      | Copy data between CPU and GPU                         |
| <code>cudaFree</code>        | Free GPU memory                                       |

#### 2.3 Matrix Multiplication

For two matrices  $A$  and  $B$  of size  $N \times N$ , the result matrix  $C$  is:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

In simple terms: **Row of A**  $\times$  **Column of B** = **One element of C**

### 2.3.1 Example (4×4 Matrices)

The code uses a for loop to fill matrix values using formulas:

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        A[i][j] = i + j;    // Row index + Column index
        B[i][j] = i * j;    // Row index * Column index
    }
}
```

**How Matrix A is filled** ( $A[i][j] = i + j$ ):

|     | j=0   | j=1   | j=2   | j=3   |
|-----|-------|-------|-------|-------|
| i=0 | 0+0=0 | 0+1=1 | 0+2=2 | 0+3=3 |
| i=1 | 1+0=1 | 1+1=2 | 1+2=3 | 1+3=4 |
| i=2 | 2+0=2 | 2+1=3 | 2+2=4 | 2+3=5 |
| i=3 | 3+0=3 | 3+1=4 | 3+2=5 | 3+3=6 |

**How Matrix B is filled** ( $B[i][j] = i \times j$ ):

|     | j=0   | j=1   | j=2   | j=3   |
|-----|-------|-------|-------|-------|
| i=0 | 0×0=0 | 0×1=0 | 0×2=0 | 0×3=0 |
| i=1 | 1×0=0 | 1×1=1 | 1×2=2 | 1×3=3 |
| i=2 | 2×0=0 | 2×1=2 | 2×2=4 | 2×3=6 |
| i=3 | 3×0=0 | 3×1=3 | 3×2=6 | 3×3=9 |

**Resulting Matrices:**

$$A = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 3 & 4 & 5 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 \\ 0 & 2 & 4 & 6 \\ 0 & 3 & 6 & 9 \end{bmatrix}$$

**Note:** These are **not random values**. The for loop generates fixed, predictable values using the formula. This makes it easy to verify the output manually.

**Calculating C[0][1]:**

$$\begin{aligned} C[0][1] &= A[0][0] \times B[0][1] + A[0][1] \times B[1][1] + A[0][2] \times B[2][1] + A[0][3] \times B[3][1] \\ &= 0 \times 0 + 1 \times 1 + 2 \times 2 + 3 \times 3 \\ &= 0 + 1 + 4 + 9 = 14 \end{aligned}$$

## 2.4 How CUDA Matrix Multiplication Works

1. Each element  $C[i][j]$  is computed **independently**.
2. Assign one **GPU thread** to compute each element.
3. For a  $4 \times 4$  matrix: launch  $4 \times 4 = 16$  threads.

4. All 16 threads compute their elements **simultaneously**!

Thread Assignment (4x4 matrix):

```
Thread(0,0) -> C[0][0]      Thread(0,1) -> C[0][1]  ...
Thread(1,0) -> C[1][0]      Thread(1,1) -> C[1][1]  ...
Thread(2,0) -> C[2][0]      Thread(2,1) -> C[2][1]  ...
Thread(3,0) -> C[3][0]      Thread(3,1) -> C[3][1]  ...
```

All 16 threads run AT THE SAME TIME on GPU!

## 2.5 Program Execution Flow

```
Step 1: Initialize matrices A, B on CPU (Host)
      |
Step 2: Allocate memory on GPU (cudaMalloc)
      |
Step 3: Copy A, B from CPU -> GPU (cudaMemcpy)
      |
Step 4: Launch Kernel on GPU <<<blocks, threads>>>
      (16 threads compute C simultaneously)
      |
Step 5: Copy result C from GPU -> CPU (cudaMemcpy)
      |
Step 6: Print result on CPU
      |
Step 7: Free GPU memory (cudaFree)
```

## 3 Code

Listing 1: Matrix Multiplication using CUDA C

```
1 #include <stdio.h>
2
3 #define N 4    // Matrix size
4
5 // CUDA Kernel - runs on GPU
6 // __global__ means this function is called from CPU, runs on GPU
7 __global__ void matrixMul(int A[N][N], int B[N][N], int C[N][N]) {
8     int row = threadIdx.y;    // Which row am I handling?
9     int col = threadIdx.x;    // Which column am I handling?
10
11     int sum = 0;
12
13     // Multiply row of A with column of B
14     for (int k = 0; k < N; k++) {
15         sum += A[row][k] * B[k][col];
16     }
17
18     C[row][col] = sum;        // Store result
19 }
```

```
20
21 int main() {
22     int A[N][N], B[N][N], C[N][N]; // Matrices on CPU (Host)
23
24     // Step 1: Initialize matrices on CPU
25     for (int i = 0; i < N; i++) {
26         for (int j = 0; j < N; j++) {
27             A[i][j] = i + j; // A[i][j] = i + j
28             B[i][j] = i * j; // B[i][j] = i * j
29         }
30     }
31
32     // GPU pointers
33     int (*d_A)[N], (*d_B)[N], (*d_C)[N];
34
35     // Step 2: Allocate memory on GPU (Device)
36     cudaMalloc((void**)&d_A, sizeof(int) * N * N);
37     cudaMalloc((void**)&d_B, sizeof(int) * N * N);
38     cudaMalloc((void**)&d_C, sizeof(int) * N * N);
39
40     // Step 3: Copy data from CPU to GPU
41     cudaMemcpy(d_A, A, sizeof(int)*N*N, cudaMemcpyHostToDevice);
42     cudaMemcpy(d_B, B, sizeof(int)*N*N, cudaMemcpyHostToDevice);
43
44     // Step 4: Launch kernel with N*N threads
45     dim3 threadsPerBlock(N, N); // 4x4 = 16 threads
46     matrixMul<<<1, threadsPerBlock>>>(d_A, d_B, d_C);
47
48     // Step 5: Copy result from GPU to CPU
49     cudaMemcpy(C, d_C, sizeof(int)*N*N, cudaMemcpyDeviceToHost);
50
51     // Step 6: Print result
52     printf("Result Matrix:\n");
53     for (int i = 0; i < N; i++) {
54         for (int j = 0; j < N; j++) {
55             printf("%d ", C[i][j]);
56         }
57         printf("\n");
58     }
59
60     // Step 7: Free GPU memory
61     cudaFree(d_A);
62     cudaFree(d_B);
63     cudaFree(d_C);
64
65     return 0;
66 }
```

## 4 Code Explanation

### 4.1 Kernel Function: `matrixMul()`

- `__global__` — tells compiler this function runs on **GPU**, called from **CPU**.
- `threadIdx.y` — gives the row number for this thread.
- `threadIdx.x` — gives the column number for this thread.
- Each thread computes **one element** of result matrix C.
- The `for` loop multiplies row of A with column of B and sums them.

### 4.2 Main Function Steps

| Step | Function                                        | Purpose                      |
|------|-------------------------------------------------|------------------------------|
| 1    | Array initialization                            | Create matrices A, B on CPU  |
| 2    | <code>cudaMalloc()</code>                       | Allocate memory on GPU       |
| 3    | <code>cudaMemcpy(HostToDevice)</code>           | Copy A, B from CPU → GPU     |
| 4    | <code>&lt;&lt;&lt;1, threads&gt;&gt;&gt;</code> | Launch kernel (16 threads)   |
| 5    | <code>cudaMemcpy(DeviceToHost)</code>           | Copy result C from GPU → CPU |
| 6    | <code>printf()</code>                           | Print result matrix          |
| 7    | <code>cudaFree()</code>                         | Free GPU memory              |

### 4.3 What does `<<<1, threadsPerBlock>>>` mean?

- 1 = number of blocks (1 block)
- `threadsPerBlock = dim3(4, 4) = 16` threads in a 4×4 grid
- Total threads =  $1 \times 16 = 16$  (one for each element of C)

## 5 Output

```
Result Matrix:
0 14 28 42
0 20 40 60
0 26 52 78
0 32 64 96
```

Verification of `C[1][2]`:

$$\begin{aligned}
 C[1][2] &= 1 \times 0 + 2 \times 2 + 3 \times 4 + 4 \times 6 \\
 &= 0 + 4 + 12 + 24 = \mathbf{40}
 \end{aligned}$$

## 6 Where to Run CUDA Programs

CUDA programs require an **NVIDIA GPU**. Here are the options:

| Platform     | GPU Required?     | How to Access                                                             |
|--------------|-------------------|---------------------------------------------------------------------------|
| College Lab  | NVIDIA GPU system | Direct access                                                             |
| Google Colab | Free GPU (cloud)  | <a href="https://colab.research.google.com">colab.research.google.com</a> |
| Personal PC  | NVIDIA GPU needed | Install CUDA Toolkit                                                      |

## 6.1 Running on Google Colab (Easiest Method – No GPU on PC needed)

1. Open browser → go to `colab.research.google.com`
2. Click **New Notebook**
3. Click **Runtime** → **Change runtime type** → **T4 GPU** → **Save**
4. **Cell 1:** Type the following (this writes the CUDA file):

Listing 2: Google Colab – Cell 1: Write CUDA file

```
%%writefile matrix_mul.cu
#include <stdio.h>
#define N 4
__global__ void matrixMul(int A[N][N], int B[N][N], int C[N][N]) {
    int row = threadIdx.y;
    int col = threadIdx.x;
    int sum = 0;
    for (int k = 0; k < N; k++)
        sum += A[row][k] * B[k][col];
    C[row][col] = sum;
}
int main() {
    int A[N][N], B[N][N], C[N][N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = i + j;
            B[i][j] = i * j;
        }
    int (*d_A)[N], (*d_B)[N], (*d_C)[N];
    cudaMalloc((void**)&d_A, sizeof(int)*N*N);
    cudaMalloc((void**)&d_B, sizeof(int)*N*N);
    cudaMalloc((void**)&d_C, sizeof(int)*N*N);
    cudaMemcpy(d_A, A, sizeof(int)*N*N, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, sizeof(int)*N*N, cudaMemcpyHostToDevice);
    dim3 threadsPerBlock(N, N);
    matrixMul<<<1, threadsPerBlock>>>(d_A, d_B, d_C);
    cudaMemcpy(C, d_C, sizeof(int)*N*N, cudaMemcpyDeviceToHost);
    printf("Result Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf("%d ", C[i][j]);
        printf("\n");
    }
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

5. **Cell 2:** Compile and run:

## Listing 3: Google Colab – Cell 2: Compile and Run

```
!nvcc matrix_mul.cu -o matrix_mul
!./matrix_mul
```

**Expected Output on Colab:**

```
Result Matrix:
0 14 28 42
0 20 40 60
0 26 52 78
0 32 64 96
```

## 6.2 Running on Ubuntu (with NVIDIA GPU)

```
# Install CUDA Toolkit
sudo apt install nvidia-cuda-toolkit

# Compile
nvcc matrix_mul.cu -o matrix_mul

# Run
./matrix_mul
```

## 6.3 Check if GPU is Available

```
nvidia-smi
# If output shows GPU info -> GPU available
# If error -> No NVIDIA GPU
```

## 7 CPU vs GPU Matrix Multiplication

| Feature         | CPU                       | GPU (CUDA)                         |
|-----------------|---------------------------|------------------------------------|
| Computation     | Sequential (nested loops) | Parallel (all elements at once)    |
| Threads         | 1 (or few)                | $N \times N$ (one per element)     |
| Time Complexity | $O(N^3)$                  | $O(N^3/P)$ where $P$ = threads     |
| Best for        | Small matrices            | Large matrices                     |
| Data Transfer   | Not needed                | CPU $\leftrightarrow$ GPU overhead |

## 8 Conclusion

In this practical, we implemented Matrix Multiplication using CUDA C. The `__global__` kernel function was used to run the computation on the GPU, where each thread computed one element of the result matrix independently. Memory was managed using `cudaMalloc`, `cudaMemcpy`, and `cudaFree`. For a  $4 \times 4$  matrix, 16 threads ran simultaneously on the GPU, demonstrating the massive parallelism capability of CUDA for matrix operations.